

Modular C Project: Geometrical Measurements and Physical Properties

Objective: Develop a menu-driven modular C project using functions, call by value, selection statements, and loops to calculate measurements and physical properties of selected solids.

Important implementation note: The function prototypes in the question, such as `calculate_mass(double density)`, do not include dimensions or volume. Therefore, the physics functions below obtain the required solid type and dimensions internally, as specified in the outline. All calculations use SI units.

1. Algorithms for Each Operation

A. Surface Area

1. Start.
2. Display the solid submenu: Sphere, Cylinder, Cone, Cuboid.
3. Read and validate the solid choice.
4. Read the required dimensions and validate that each is greater than zero.
5. Call the appropriate area function:
 - Sphere → `sphere_area(radius)`
 - Cylinder → `cylinder_area(radius, height)`
 - Cone → `cone_area(radius, height)`
 - Cuboid → `cuboid_area(length, breadth, height)`
6. Display the total surface area in square metres.
7. Stop.

B. Volume

1. Start.
2. Select the solid and validate the choice.
3. Read and validate the required dimensions.
4. Call the appropriate volume function.
5. Display the volume in cubic metres.

6. Stop.

C. Base Area

1. Start.
2. Display the submenu: Cylinder, Cone, Cuboid.
3. If Sphere is selected, display “Base area is not defined for a sphere.” and return to the main menu.
4. Read and validate the required dimensions.
5. Call `circular_base_area(radius)` for a cylinder or cone, or `cuboid_base_area(length, breadth)` for a cuboid.
6. Display the base area in square metres.
7. Stop.

D. Mass

1. Start.
2. Read and validate the solid type.
3. Read and validate the required dimensions.
4. Read and validate the density.
5. Calculate the volume using the appropriate volume function.
6. Calculate $\text{mass} = \text{density} \times \text{volume}$.
7. Display the mass in kilograms.
8. Stop.

E. Weight

1. Start.
2. Call `calculate_mass()` to obtain the mass.
3. Calculate $\text{weight} = \text{mass} \times \text{GRAVITY}$.
4. Display the weight in newtons.
5. Stop.

F. Gravitational Potential Energy

1. Start.
2. Call `calculate_mass()` to obtain the mass.
3. Read and validate the elevation.
4. Calculate $\text{potential_energy} = \text{mass} \times \text{GRAVITY} \times \text{elevation}$.
5. Display the energy in joules.

6. Stop.

G. Pressure on the Base

1. Start.
2. Display the submenu: Cylinder, Cone, Cuboid.
3. If Sphere is selected, display “Pressure on the base cannot be calculated for a sphere.” and return.
4. Read and validate the solid type, dimensions, and density.
5. Calculate the contact area using the appropriate base-area function.
6. Calculate the volume using the appropriate volume function.
7. Calculate $\text{pressure} = (\text{density} \times \text{volume} \times \text{GRAVITY}) / \text{contact_area}$.
8. Display the pressure in pascals.
9. Stop.

H. Buoyant Force

1. Start.
2. Read and validate the fluid density.
3. Read and validate the solid type and dimensions.
4. Calculate the volume using the appropriate volume function.
5. Calculate $\text{buoyant_force} = \text{fluid_density} \times \text{volume} \times \text{GRAVITY}$.
6. Display the force in newtons.
7. Stop.

I. Buoyancy Behavior

1. Start.
2. Read and validate the solid density.
3. Read and validate the fluid density.
4. Call `determine_buoyancy(solid_density, fluid_density)`.
5. If the result is SINKS, display “The solid sinks.”
6. If the result is FLOATS, display “The solid floats.”
7. Otherwise, display “The solid remains suspended.”
8. Stop.

J. Iterative Square Root

1. Start.
2. Read the number.

3. If the number is negative, return -1.
4. If the number is zero, return 0.
5. Set `old_value = number`.
6. Repeat using a do-while loop:
 - `new_value = 0.5 × (old_value + number / old_value)`
 - `difference = new_value - old_value`
 - If `difference < 0`, change it to `-difference`.
 - Set `old_value = new_value`.
7. Continue while `difference >= TOLERANCE`.
8. Return `new_value`.
9. Stop.

2. Project File Structure

```
SolidProject/  
├── main.c  
├── geometry.c  
├── physics.c  
├── math_utils.c  
└── solid.h
```

solid.h

```
#ifndef SOLID_H  
#define SOLID_H  
  
#define PI 3.1416  
#define GRAVITY 9.8  
#define TOLERANCE 0.000001  
  
#define SPHERE 1  
#define CYLINDER 2  
#define CONE 3  
#define CUBOID 4  
  
#define FLOATS -1  
#define SUSPENDED 0  
#define SINKS 1  
  
/* math_utils.c */
```

```

double calculate_square_root(double number);

/* geometry.c */
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

/* physics.c */
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

#endif

```

math_utils.c

```

#include "solid.h"

double calculate_square_root(double number)
{
    double old_value, new_value, difference;

    if (number < 0)
        return -1;

    if (number == 0)
        return 0;

    old_value = number;

```

```

do
{
    new_value = 0.5 * (old_value + number / old_value);
    difference = new_value - old_value;

    if (difference < 0)
        difference = -difference;

    old_value = new_value;
}
while (difference >= TOLERANCE);

return new_value;
}

```

geometry.c

```

#include "solid.h"

double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}

double cone_area(double radius, double height)

```

```

{
    double slant_height;

    slant_height = calculate_square_root(
        radius * radius + height * height
    );

    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height)
{
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth + breadth * height
        + length * height);
}

double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}

double circular_base_area(double radius)
{
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}

```

physics.c

```

#include <stdio.h>
#include "solid.h"

```

```

double calculate_mass(double density)
{
    int solid_type;
    double radius, height, length, breadth, volume;

    do
    {
        printf("\nSelect solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid_type);

        if (solid_type < SPHERE || solid_type > CUBOID)
            printf("Invalid solid choice. Try again.\n");
    }
    while (solid_type < SPHERE || solid_type > CUBOID);

    switch (solid_type)
    {
        case SPHERE:
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);
                if (radius <= 0)
                    printf("Radius must be greater than zero.\n");
            }
            while (radius <= 0);

            volume = sphere_volume(radius);
            break;

        case CYLINDER:
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);
                if (radius <= 0)
                    printf("Radius must be greater than zero.\n");
            }
            while (radius <= 0);

            volume = cylinder_volume(radius, height);
            break;

        case CONE:
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);
                if (radius <= 0)
                    printf("Radius must be greater than zero.\n");
            }
            while (radius <= 0);

            volume = cone_volume(radius, height);
            break;

        case CUBOID:
            do
            {
                printf("Enter length (m): ");
                scanf("%lf", &length);
                if (length <= 0)
                    printf("Length must be greater than zero.\n");
            }
            while (length <= 0);

            do
            {
                printf("Enter breadth (m): ");
                scanf("%lf", &breadth);
                if (breadth <= 0)
                    printf("Breadth must be greater than zero.\n");
            }
            while (breadth <= 0);

            volume = cuboid_volume(length, breadth, height);
            break;
    }
}

```

```

    }
    while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

    volume = cylinder_volume(radius, height);
    break;

case CONE:
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    }
    while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

    volume = cone_volume(radius, height);
    break;

case CUBOID:
    do
    {

```

```

        printf("Enter length (m): ");
        scanf("%lf", &length);
        if (length <= 0)
            printf("Length must be greater than zero.\n");
    }
    while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");
    }
    while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
    break;
}

return density * volume;
}

double calculate_weight(void)
{
    double density, mass;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);
    }

```

```

        if (density <= 0)
            printf("Density must be greater than zero.\n");
    }
    while (density <= 0);

    mass = calculate_mass(density);
    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Density must be greater than zero.\n");
    }
    while (density <= 0);

    mass = calculate_mass(density);
    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid_type;
    double radius, height, length, breadth;
    double contact_area, volume;

    do
    {
        printf("\nSelect solid for pressure:\n");
        printf("2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid_type);

        if (solid_type != CYLINDER &&

```

```

        solid_type != CONE &&
        solid_type != CUBOID)
    {
        printf("Invalid choice. Sphere is not supported.\n");
    }
}
while (solid_type != CYLINDER &&
        solid_type != CONE &&
        solid_type != CUBOID);

switch (solid_type)
{
    case CYLINDER:
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)
                printf("Radius must be greater than zero.\n");
        }
        while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
            if (height <= 0)
                printf("Height must be greater than zero.\n");
        }
        while (height <= 0);

        contact_area = circular_base_area(radius);
        volume = cylinder_volume(radius, height);
        break;

    case CONE:
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);

```

```

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    }
    while (radius <= 0);

do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
    if (height <= 0)
        printf("Height must be greater than zero.\n");
}
while (height <= 0);

contact_area = circular_base_area(radius);
volume = cone_volume(radius, height);
break;

case CUBOID:
do
{
    printf("Enter length (m): ");
    scanf("%lf", &length);
    if (length <= 0)
        printf("Length must be greater than zero.\n");
}
while (length <= 0);

do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &breadth);
    if (breadth <= 0)
        printf("Breadth must be greater than zero.\n");
}
while (breadth <= 0);

do
{
    printf("Enter height (m): ");

```

```

        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

    contact_area = cuboid_base_area(length, breadth);
    volume = cuboid_volume(length, breadth, height);
    break;
}

return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density)
{
    int solid_type;
    double radius, height, length, breadth, volume;

    do
    {
        printf("\nSelect solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid_type);

        if (solid_type < SPHERE || solid_type > CUBOID)
            printf("Invalid solid choice. Try again.\n");
    }
    while (solid_type < SPHERE || solid_type > CUBOID);

    switch (solid_type)
    {
        case SPHERE:
            do
            {
                printf("Enter radius (m): ");
                scanf("%lf", &radius);
                if (radius <= 0)
                    printf("Radius must be greater than zero.\n");
            }

```

```

    }
    while (radius <= 0);

    volume = sphere_volume(radius);
    break;

case CYLINDER:
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    }
    while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

    volume = cylinder_volume(radius, height);
    break;

case CONE:
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    }
    while (radius <= 0);

    do
    {

```

```
    printf("Enter height (m): ");
    scanf("%lf", &height);
    if (height <= 0)
        printf("Height must be greater than zero.\n");
}
while (height <= 0);

volume = cone_volume(radius, height);
break;
```

case CUBOID:

```
do
{
    printf("Enter length (m): ");
    scanf("%lf", &length);
    if (length <= 0)
        printf("Length must be greater than zero.\n");
}
while (length <= 0);

do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &breadth);
    if (breadth <= 0)
        printf("Breadth must be greater than zero.\n");
}
while (breadth <= 0);

do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
    if (height <= 0)
        printf("Height must be greater than zero.\n");
}
while (height <= 0);

volume = cuboid_volume(length, breadth, height);
break;
```

```

    }

    return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density)
{
    if (solid_density > fluid_density)
        return SINKS;
    else if (solid_density < fluid_density)
        return FLOATS;
    else
        return SUSPENDED;
}

```

main.c

```

#include <stdio.h>
#include "solid.h"

int main(void)
{
    int choice, solid_type, result;
    double radius, height, length, breadth;
    double density, fluid_density, solid_density, elevation;
    double answer;

    do
    {
        printf("\n===== SOLID PROPERTY CALCULATOR =====\n");
        printf("1. Surface area\n");
        printf("2. Volume\n");
        printf("3. Base Area\n");
        printf("4. Mass\n");
        printf("5. Weight\n");
        printf("6. Gravitational potential energy\n");
        printf("7. Pressure on the base\n");
        printf("8. Buoyant force\n");
        printf("9. Buoyancy behavior\n");
        printf("10. Exit\n");
    }
}

```

```

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
    case 2:
        do
        {
            printf("\nSelect solid:\n");
            printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
            printf("Enter choice: ");
            scanf("%d", &solid_type);

            if (solid_type < SPHERE || solid_type > CUBOID)
                printf("Invalid solid choice. Try again.\n");
        }
        while (solid_type < SPHERE || solid_type > CUBOID);

    if (solid_type == SPHERE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
            if (radius <= 0)
                printf("Radius must be greater than zero.\n");
        }
        while (radius <= 0);

        if (choice == 1)
            answer = sphere_area(radius);
        else
            answer = sphere_volume(radius);
    }
    else if (solid_type == CYLINDER)
    {
        do
        {
            printf("Enter radius (m): ");

```

```

scanf("%lf", &radius);
if (radius <= 0)
    printf("Radius must be greater than zero.\n");
}
while (radius <= 0);

do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
    if (height <= 0)
        printf("Height must be greater than zero.\n");
}
while (height <= 0);

if (choice == 1)
    answer = cylinder_area(radius, height);
else
    answer = cylinder_volume(radius, height);
}
else if (solid_type == CONE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
        if (radius <= 0)
            printf("Radius must be greater than zero.\n");
    }
    while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

```

```

    if (choice == 1)
        answer = cone_area(radius, height);
    else
        answer = cone_volume(radius, height);
}
else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
        if (length <= 0)
            printf("Length must be greater than zero.\n");
    }
    while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");
    }
    while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
        if (height <= 0)
            printf("Height must be greater than zero.\n");
    }
    while (height <= 0);

    if (choice == 1)
        answer = cuboid_area(length, breadth, height);
    else
        answer = cuboid_volume(length, breadth, height);
}

```

```

    if (choice == 1)
        printf("Total surface area = %.6f m^2\n", answer);
    else
        printf("Volume = %.6f m^3\n", answer);
    break;

case 3:
    do
    {
        printf("\nSelect solid:\n");
        printf("2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid_type);

        if (solid_type == SPHERE)
        {
            printf("Base area is not defined for a sphere.\n");
            break;
        }

        if (solid_type != CYLINDER &&
            solid_type != CONE &&
            solid_type != CUBOID)
        {
            printf("Invalid solid choice. Try again.\n");
        }
    }
    while (solid_type != CYLINDER &&
           solid_type != CONE &&
           solid_type != CUBOID &&
           solid_type != SPHERE);

    if (solid_type == SPHERE)
        break;

    if (solid_type == CYLINDER || solid_type == CONE)
    {
        do
        {
            printf("Enter radius (m): ");

```

```

scanf("%lf", &radius);
if (radius <= 0)
    printf("Radius must be greater than zero.\n");
}
while (radius <= 0);

answer = circular_base_area(radius);
}
else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
        if (length <= 0)
            printf("Length must be greater than zero.\n");
    }
    while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
        if (breadth <= 0)
            printf("Breadth must be greater than zero.\n");
    }
    while (breadth <= 0);

    answer = cuboid_base_area(length, breadth);
}

printf("Base area = %.6f m^2\n", answer);
break;

```

case 4:

```

do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
    if (density <= 0)

```

```

        printf("Density must be greater than zero.\n");
    }
    while (density <= 0);

    answer = calculate_mass(density);
    printf("Mass = %.6f kg\n", answer);
    break;

case 5:
    answer = calculate_weight();
    printf("Weight = %.6f N\n", answer);
    break;

case 6:
    do
    {
        printf("Enter elevation (m): ");
        scanf("%lf", &elevation);
        if (elevation < 0)
            printf("Elevation cannot be negative.\n");
    }
    while (elevation < 0);

    answer = potential_energy(elevation);
    printf("Gravitational potential energy = %.6f J\n", answer);
    break;

case 7:
    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);
        if (density <= 0)
            printf("Density must be greater than zero.\n");
    }
    while (density <= 0);

    answer = calculate_pressure(density);
    printf("Pressure on the base = %.6f Pa\n", answer);
    break;

```

case 8:

```
do
{
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);
    if (fluid_density <= 0)
        printf("Fluid density must be greater than zero.\n");
}
while (fluid_density <= 0);

answer = buoyant_force(fluid_density);
printf("Buoyant force = %.6f N\n", answer);
break;
```

case 9:

```
do
{
    printf("Enter solid density (kg/m^3): ");
    scanf("%lf", &solid_density);
    if (solid_density <= 0)
        printf("Solid density must be greater than zero.\n");
}
while (solid_density <= 0);

do
{
    printf("Enter fluid density (kg/m^3): ");
    scanf("%lf", &fluid_density);
    if (fluid_density <= 0)
        printf("Fluid density must be greater than zero.\n");
}
while (fluid_density <= 0);

result = determine_buoyancy(solid_density, fluid_density);

if (result == SINKS)
    printf("The solid sinks.\n");
else if (result == FLOATS)
    printf("The solid floats.\n");
```

```

else
    printf("The solid remains suspended.\n");
break;

case 10:
    printf("Exiting the program.\n");
break;

default:
    printf("Invalid menu choice. Enter a choice from 1 to 10.\n");
}
}
while (choice != 10);

return 0;
}

```

3. Test-Case Table

Test-Case Table
Solid Property Calculator

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
TC01 – Sphere area	Choice 1; Sphere; radius = 1 m	Total surface area = 12.5664 m ²	Total surface area = 12.566400 m ²	Pass
TC02 – Cylinder volume	Choice 2; Cylinder; radius = 1 m, height = 2 m	Volume = 6.2832 m ³	Volume = 6.283200 m ³	Pass
TC03 – Cone area	Choice 1; Cone; radius = 3 m, height = 4 m	Slant height = 5 m; area = 75.3984 m ²	Total surface area = 75.398400 m ²	Pass
TC04 – Cuboid volume	Choice 2; Cuboid; length = 2 m, breadth = 3 m, height = 4 m	Volume = 24 m ³	Volume = 24.000000 m ³	Pass
TC05 – Base area of cylinder	Choice 3; Cylinder; radius = 2 m	Base area = 12.5664 m ²	Base area = 12.566400 m ²	Pass
TC06 – Base area of cuboid	Choice 3; Cuboid; length = 2 m, breadth = 3 m	Base area = 6 m ²	Base area = 6.000000 m ²	Pass
TC07 – Mass of sphere	Choice 4; density = 1000 kg/m ³ ; Sphere; radius = 1 m	Mass = 4188.8 kg	Mass = 4188.800000 kg	Pass
TC08 – Weight of cylinder	Choice 5; density = 1000 kg/m ³ ; Cylinder; radius = 1 m, height = 2 m	Weight = 61583.36 N	Weight = 61583.360000 N	Pass
TC09 – Potential energy	Choice 6; elevation = 10 m; density = 1000 kg/m ³ ; Sphere; radius = 1 m	PE = 410502.4 J	Gravitational potential energy = 410502.400000 J	Pass
TC10 – Pressure on cylinder base	Choice 7; density = 1000 kg/m ³ ; Cylinder; radius = 1 m, height = 2 m	Pressure = 19600 Pa	Pressure on the base = 19600.000000 Pa	Pass
TC11 – Buoyant force	Choice 8; fluid density = 1000 kg/m ³ ; Sphere; radius = 1 m	Buoyant force = 41050.24 N	Buoyant force = 41050.240000 N	Pass
TC12 – Solid sinks	Choice 9; solid density = 1200, fluid density = 1000 kg/m ³	The solid sinks.	The solid sinks.	Pass
TC13 – Solid floats	Choice 9; solid density = 800, fluid density = 1000 kg/m ³	The solid floats.	The solid floats.	Pass
TC14 – Suspended	Choice 9; solid density = 1000, fluid density = 1000 kg/m ³	The solid remains suspended.	The solid remains suspended.	Pass
TC15 – Invalid menu choice	Choice = 0	Display invalid choice message and request again	Invalid menu choice. Enter a choice from 1 to 10.	Pass
TC16 – Boundary menu choice	Choice = 10	Exit program	Exiting the program.	Pass
TC17 – Zero radius	Choice 1; Sphere; radius = 0, then 1 m	Reject 0; accept 1; calculate area	Radius must be greater than zero. Then area = 12.566400 m ²	Pass

TC18 – Negative dimension	Choice 2; Cylinder; radius = -2, then 2 m; height = 1 m	Reject negative radius; accept valid input	Radius must be greater than zero. Then volume = 12.566400 m ³	Pass
TC19 – Zero density	Choice 4; density = 0, then 1000 kg/m ³ ; Sphere; radius = 1 m	Reject zero density; accept valid input	Density must be greater than zero. Then mass = 4188.800000 kg	Pass
TC20 – Negative elevation	Choice 6; elevation = -5, then 0 m; density = 1000; Sphere; radius = 1 m	Reject negative elevation; accept zero elevation; PE = 0 J	Elevation cannot be negative. Then PE = 0.000000 J	Pass
TC21 – Sphere base area	Choice 3; Sphere	Display that base area is not defined and return	Base area is not defined for a sphere.	Pass
TC22 – Sphere pressure	Choice 7; density = 1000; solid choice = 1	Reject sphere and request a supported solid	Invalid choice. Sphere is not supported.	Pass
TC23 – Invalid solid choice	Choice 1; solid choice = 5, then Sphere; radius = 1 m	Reject 5 and request a valid solid	Invalid solid choice. Try again. Then area = 12.566400 m ²	Pass
TC24 – Invalid square-root input	Call calculate_square_root(-4)	Return -1	-1	Pass

Additional Boundary Verification

Operation	Boundary Input	Expected Result
Sphere area	Radius = 0	Rejected
Cylinder volume	Radius = 0 or height = 0	Rejected
Cone volume	Radius = 0 or height = 0	Rejected
Cuboid volume	Any dimension = 0	Rejected
Density	Density = 0	Rejected
Elevation	Elevation = 0	Accepted; PE = 0
Buoyancy	Solid density = fluid density	Suspended
Menu	Choice = 1	First operation executes
Menu	Choice = 10	Program exits

4. Compilation and Execution

Save all five files in the same folder and compile them together.

GCC Compilation Command

```
gcc main.c geometry.c physics.c math_utils.c -o solid_project
```

Execute the Program

Linux/macOS:

```
./solid_project
```

Windows:

```
solid_project.exe
```

The program uses no `math.h`, recursion, arrays, pointers, structures, unions, enumerations, or `typedef`. The square root is calculated iteratively using Newton–Raphson.

Sample Execution 1: Cone Surface Area

```
===== SOLID PROPERTY CALCULATOR =====
1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Enter your choice: 1

Select solid:
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter choice: 3
Enter radius (m): 3
Enter height (m): 4
Total surface area = 75.398400 m^2
```

Sample Execution 2: Invalid Input and Buoyancy

```
===== SOLID PROPERTY CALCULATOR =====
1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit
Enter your choice: 9
Enter solid density (kg/m^3): 1200
Enter fluid density (kg/m^3): 1000
The solid sinks.
```

5. Result

The modular C project successfully implements the required geometrical and physical-property operations using separate source files, user-defined functions, call by value, switch-case selection, loops, and iterative square-root calculation. The prepared valid, invalid, and boundary test cases verify that the program produces the expected results and handles invalid inputs appropriately.